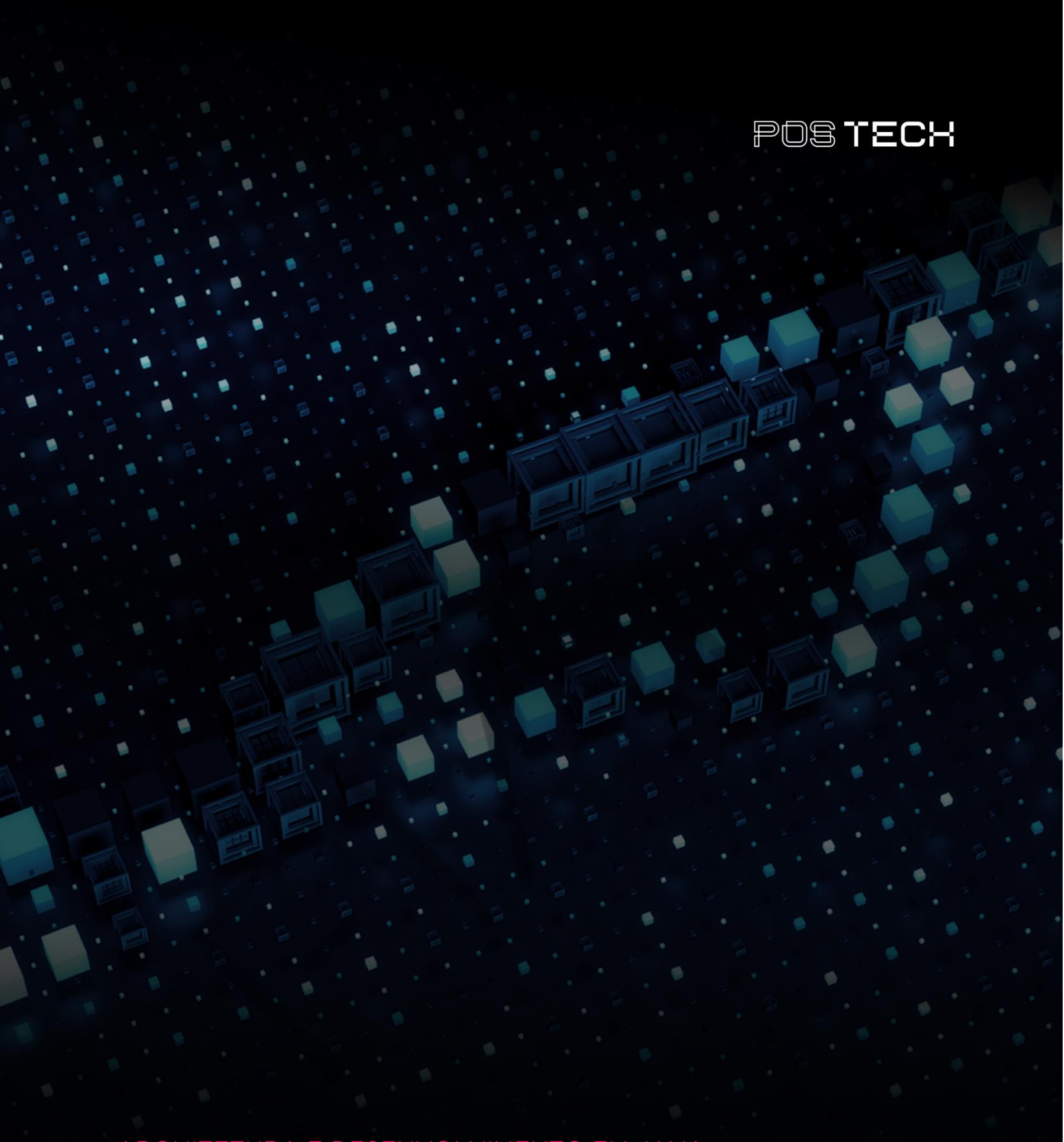


PÓS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 02 -

AUTENTICAÇÃO E AUTORIZAÇÃO

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 11
REFERÊNCIAS 12
PALAVRAS-CHAVE 13

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Nessa aula, veremos as diferenças entre autenticação e autorização para a segurança em aplicações Java com Spring Security. Implementaremos a autenticação e autorização OAuth2 com integração ao Google e, por fim, implementaremos dois modelos de autenticação e autorização Stateless JWT em uma API REST.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Os projetos que criaremos aqui serão importantes para alavancar seu conhecimento sobre a segurança das aplicações e garantir sua aplicação prática. Veremos como autenticar e autorizar o acesso a APIs através de um Identity Provider e seus modelos de implementação.

Falaremos sobre os principais desafios na segurança das aplicações Java e como isso é vital para o sucesso do projeto de software que estamos implementando. Além disso, demostraremos a implementação de um sistema para autenticação e autorização OAuth 2.0 com integração ao Google e implementaremos dois modelos de autenticação e autorização Stateless JWT em uma API REST.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Você sabe qual é a diferença entre autenticação e autorização? Vamos entender utilizando como exemplo um festival de música.

Pense que a **autenticação** é o momento em que você chega na entrada do evento e mostra o seu ingresso. Aqui, a equipe de segurança verifica se você é quem diz ser e se realmente possui um ingresso válido para entrar. Eles conferem sua identidade, ou seja, quem você é. No mundo da tecnologia, isso seria como fazer um login, inserindo seu usuário e senha para confirmar que você é você mesmo.

Agora, a **autorização** é o que acontece depois que você já entrou no evento. Ela define até onde você pode ir e o que pode fazer dentro do evento. Por exemplo, se você tem um ingresso **VIP**, você pode ter acesso a áreas exclusivas, como o backstage ou lounges especiais. Se você tem um ingresso **COMUM**, só pode circular pelas áreas gerais. No contexto da tecnologia, isso seria, por exemplo, decidir quais páginas ou funcionalidades do sistema você pode acessar depois de fazer login.

Resumindo, **autenticação** verifica quem você é e **autorização** verifica o que você pode fazer!

Agora, vamos observar alguns conceitos importantes, mencionados nas videoaulas. O primeiro é o de **fluxo de autenticação** em nossas aplicações.

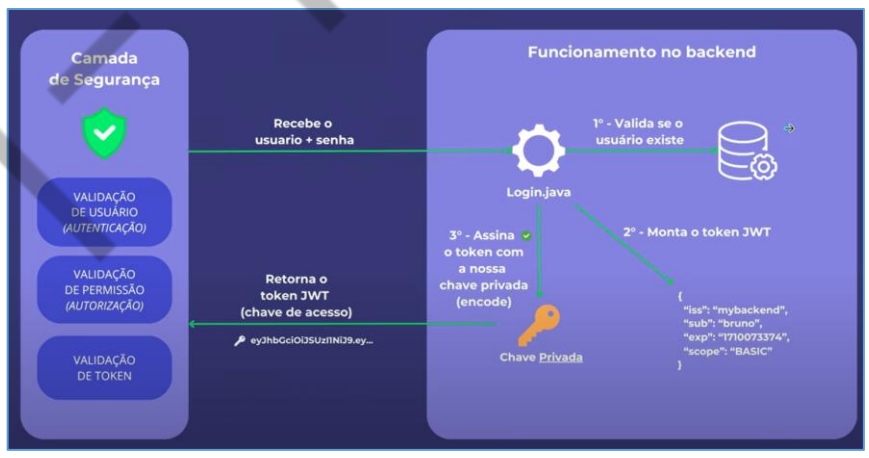


Figura 1 – Fluxo de Autenticação
Fonte: elaborado pelo autor (2025)

A figura 1 descreve um fluxo de autenticação e autorização baseado em JWT (JSON Web Token) no backend.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

A **camada de segurança** é composta por três etapas principais:

- Validação de usuário (autenticação): verifica se as credenciais (usuário e senha) fornecidas são válidas.
- Validação de permissão (autorização): confirma se o usuário autenticado tem permissão para acessar um recurso ou funcionalidade.
- Validação de token: garante que o token JWT fornecido em solicitações subsequentes seja válido e não tenha sido alterado.

O **funcionamento no Backend** é o seguinte:

- Recebe usuário + senha: o backend recebe as credenciais fornecidas pelo cliente (geralmente via formulário de login).
- Valida se o usuário existe: o sistema verifica, em um banco de dados ou outro repositório de usuários, se o nome de usuário e a senha fornecidos correspondem a um registro válido.
- Monta o token JWT: se as credenciais forem válidas, o backend cria um token JWT contendo:
    - iss (Issuer): identifica quem emitiu o token (por exemplo, mybackend).
    - sub (Subject): identifica o usuário (por exemplo, bruno).
    - exp (Expiration): define quando o token expira (timestamp).
    - scope: representa as permissões do usuário (neste caso, BASIC).
- Assina o token com chave privada: o token é assinado com uma chave privada para garantir que ele não possa ser modificado por terceiros. A assinatura também serve para verificar a autenticidade do token.
- Retorna o token JWT: o backend envia o token JWT gerado para o cliente, que o usará para acessar recursos protegidos no sistema.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Temos alguns pontos importantes no fluxo. O token JWT serve como uma "chave de acesso" e será enviado em cada requisição subsequente no cabeçalho HTTP (geralmente no campo Authorization). O backend usará a chave pública correspondente à chave privada para validar a assinatura do token, garantindo que ele não foi adulterado. Após o token expirar (campo exp), o cliente precisará solicitar um novo token.

O segundo é o **fluxo de autorização** em nossas aplicações.

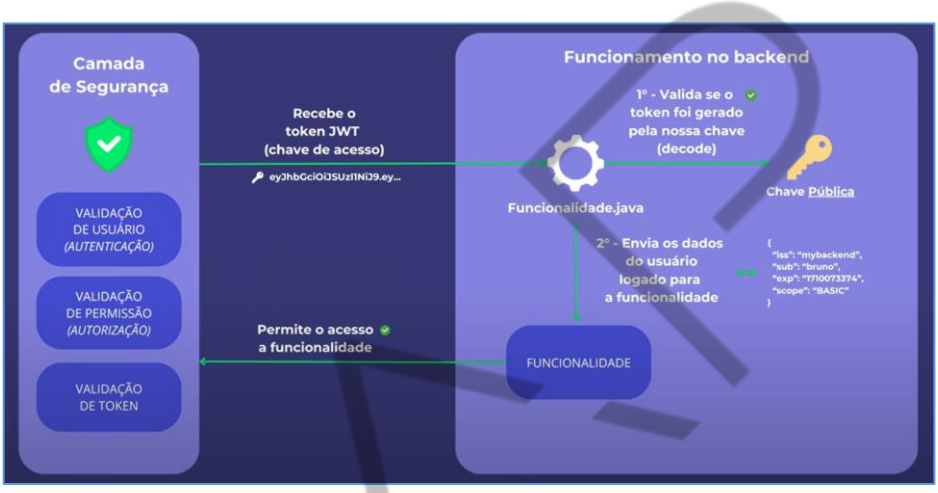


Figura 2 – Fluxo de Autorização
Fonte: elaborado pelo autor (2025)

A figura 2 representa o fluxo de validação e autorização de um token JWT para acesso a uma funcionalidade no backend.

A **camada de segurança** realiza três tipos de validações. Na **validação de usuário (autenticação)**, verificamos se o usuário é quem diz ser, por exemplo, se já foi autenticado previamente e possui um token válido). Na **validação de permissão (autorização)**, confirmamos se o usuário tem os privilégios necessários para acessar a funcionalidade solicitada. Por fim, na **validação de token**, garantimos que o token JWT enviado na requisição é válido, não foi alterado e está dentro do período de validade.

Já o **funcionamento no backend** primeiramente recebe o token JWT (chave de acesso). O cliente envia o token JWT no cabeçalho da requisição (normalmente no campo Authorization com o formato Bearer <token>). Então, ele valida se o token foi gerado pela chave privada (decode), utilizando a chave pública correspondente à chave privada usada para assinar o token. Ele faz o "decode" (decodificação) para

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

verificar a integridade do token (se não foi modificado) e a validade do token (se não expirou, conforme o campo exp).

Se o token for válido, o backend extrai as informações contidas nele, como:

- iss (quem emitiu o token, ex.: **mybackend**);
- sub (usuário associado ao token, ex.: **bruno**);
- scope (permissões do usuário, ex.: **BASIC**).

Após a validação bem-sucedida, os dados do token (como sub e scope) são repassados para a funcionalidade que o usuário deseja acessar. Esses dados podem ser usados para personalizar a experiência ou limitar as ações com base nas permissões. Se todas as validações forem bem-sucedidas e o usuário tiver as permissões necessárias, o backend concede acesso à funcionalidade solicitada.

Vale destacar nesse fluxo que o uso da **chave pública** garante que somente tokens assinados pelo servidor sejam aceitos, impedindo tokens falsificados, e a validação do scope assegura que o acesso seja controlado com base nas permissões do usuário. Esse processo é feito para toda requisição a recursos protegidos, mantendo a aplicação segura e o controle de acesso eficiente.

Diferenças entre JWT e OAuth 2.0 e seus métodos de implementação

O **JWT (JSON Web Token)** é um **padrão aberto para criar tokens de acesso** compactos e seguros, que permitem a troca de informações entre duas partes. Um JWT é composto por três partes: **cabeçalho**, **payload** e **assinatura**, codificados em **base64**.

É utilizado amplamente para autenticação e autorização em APIs, onde o token é passado no cabeçalho HTTP como Bearer token.

Já **OAuth 2.0** é um **protocolo de autorização Open ID** que permite que um serviço conceda acesso a um recurso seguro em nome de um usuário. Ele utiliza tokens de acesso e, opcionalmente, tokens de atualização, para manter sessões seguras sem expor as credenciais do usuário.

Pode ser implementado em conjunto com JWT, onde os tokens de acesso são JWTs assinados, ou sem ele, dependendo do sistema de autenticação.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Em resumo, **JWT** é o formato do token, enquanto **OAuth 2.0** é um protocolo de autorização que pode utilizar tokens **JWT** para autenticar e autorizar acesso a recursos.

Veremos agora alguns conceitos importantes para criação de um projeto que realize a autenticação utilizando o **OAuth 2.0** e um **Identity Provider** (provedor de identidade), nesse caso, o **Google**.

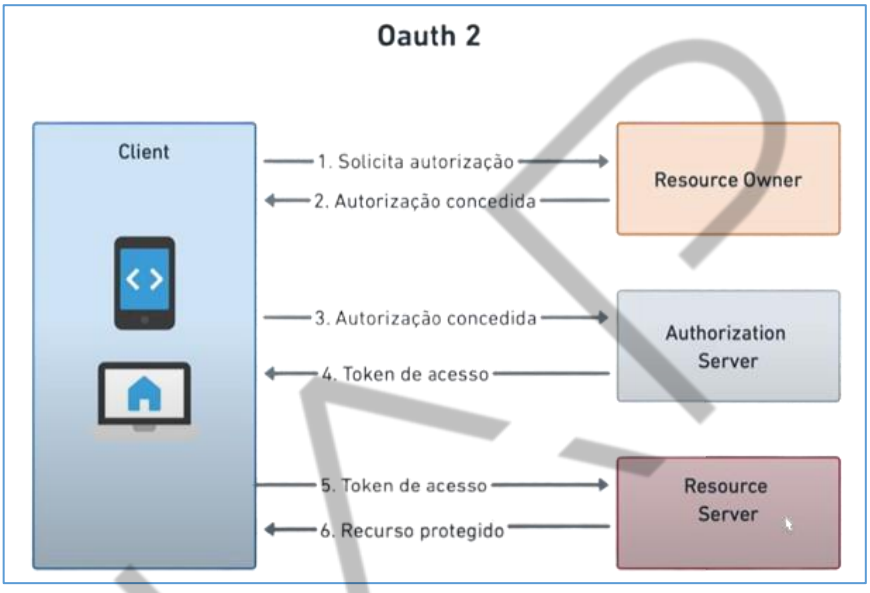


Figura 3 – Fluxo OAuth 2.0
Fonte: elaborado pelo autor (2025)

Resource Owner (proprietário do recurso): é o usuário que possui os dados ou recursos protegidos. Exemplo: uma pessoa que concede permissão para um aplicativo acessar suas informações em uma rede social.

Authorization Server (servidor de autorização): é o componente responsável por autenticar o **Resource Owner** e conceder tokens de acesso aos clientes autorizados. Exemplo: o sistema de autenticação do **Google** que verifica as credenciais do usuário e emite um token.

Resource Server (servidor de recurso): é a API ou serviço que hospeda os recursos protegidos e os fornece aos clientes que apresentam um token válido. Exemplo: a API do **Google** que retorna dados do perfil do usuário, desde que o token seja válido e tenha as permissões corretas.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Outro ponto importante é a implementação de um serviço **modernizado** com autenticação **Stateless JWT e OAuth 2.0** em uma **API REST**. No nosso caso, fizemos uso do par de **chaves pública** e **privada**.

Como instalar o OpenSSL no Windows

Baixe o OpenSSL: vá até o site oficial do OpenSSL ou baixe a versão Windows de um repositório confiável, como **Shining Light Productions.**

Escolha a versão adequada: baixe a versão correta para seu sistema operacional (**32 ou 64 bits**).

Instale o OpenSSL: durante a instalação, marque a opção para adicionar o diretório de instalação do OpenSSL ao **PATH** do sistema, se disponível.

Adicione o endereço: **C:\Program Files\OpenSSL-Win64\bin**

Para gerar as chaves, digite os seguintes comandos no prompt do Windows (**CMD**):

- Gerar a chave privada: **openssl genrsa -out private_key.pem 2048**
- Gerar a chave pública: **openssl rsa -in private_key.pem -pubout -out public_key.pem**

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, discutimos os principais desafios na segurança em aplicações Java com Spring Security na versão atualizada 6.1. Navegamos pelas estruturas (classes/interfaces) do Spring Security e aprendemos a implementar a autenticação e autorização OAuth2 com integração ao Google. Vimos pontos importantes para codificação dos projetos e implementamos dois modelos de autenticação e autorização Stateless JWT em uma API REST.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

BAELGUNG. **Guia do Baeldung**. Disponível em: https://www.baeldung.com/?s=spring%20security. Acesso em: 08 fev. 2025.

NARDONE, M; SCARIONI, C. **Pro Spring Security: Securing Spring Framework 6 and Boot 3-based Java Applications**. New York: Apress; 3ª edição, 2023.

SPRING.IO. **Documentação do Spring Security**. Disponível em: https://spring.io/projects/spring-security. Acesso em: 08 fev. 2025.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Palavras-chave: spring. security. autenticação, autorização, jwt.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com